

3. Operátorok C#-ban

Mi az az operátor?

Az operátor egy olyan szimbólum, amely műveletet hajt végre egy vagy több operanduson (értéken). Az operátorok segítségével matematikai műveleteket, összehasonlításokat és logikai műveleteket végezhetünk.

Aritmetikai operátorok

Matematikai műveletek végrehajtására szolgálnak.

Alapvető műveletek

```
int a = 10;
int b = 3;

// Összeadás (+)
int osszeg = a + b;           // 13
Console.WriteLine($"{a} + {b} = {osszeg}");

// Kivonás (-)
int kulonbseg = a - b;        // 7
Console.WriteLine($"{a} - {b} = {kulonbseg}");

// Szorzás (*)
int szorzat = a * b;          // 30
Console.WriteLine($"{a} * {b} = {szorzat}");

// Osztás (/)
int hanyados = a / b;         // 3 (egész osztás!)
Console.WriteLine($"{a} / {b} = {hanyados}");

// Maradékos osztás (modulo) (%)
int maradek = a % b;          // 1 (10 = 3*3 + 1)
Console.WriteLine($"{a} % {b} = {maradek}");
```

FIGYELEM: Egész és lebegőpontos osztás

```
// Egész osztás - csonkítás történik!
int x = 10;
int y = 3;
int eredmeny1 = x / y;        // 3 (nem 3.333...)
Console.WriteLine(eredmeny1);

int z = 7 / 2;                // 3 (nem 3.5!)
Console.WriteLine(z);
```

```
// Lebegőpontos osztás
double a = 10.0;
double b = 3.0;
double eredmeny2 = a / b;           // 3.3333333333333335
Console.WriteLine(eredmeny2);

// Ha legalább egy operandus double/float, a végeredmény is az lesz
double eredmeny3 = 10 / 3.0;        // 3.3333333333333335
double eredmeny4 = 10.0 / 3;        // 3.3333333333333335
double eredmeny5 = (double)10 / 3;  // 3.3333333333333335

// Mindkettő int, akkor int marad
double eredmeny6 = 10 / 3;          // 3.0 (előbb 3 lesz, aztán konvertálódik)
```

Gyakorlati példák

```
// Páros-e egy szám?
int szam = 15;
bool paros = (szam % 2 == 0);      // false
Console.WriteLine($"{szam} páros? {paros}");

// Osztható-e 5-tel?
int szam2 = 25;
bool oszthatoOttel = (szam2 % 5 == 0); // true
Console.WriteLine($"{szam2} osztható 5-tel? {oszthatoOttel}");

// Utolsó számjegy
int szam3 = 12345;
int utolsoSzamjegy = szam3 % 10; // 5
Console.WriteLine($"Utolsó számjegy: {utolsoSzamjegy}");

// Átváltás órából percre
int orak = 2;
int percek = 30;
int osszesPerc = (orak * 60) + percek; // 150
Console.WriteLine($"{orak} óra {percek} perc = {osszesPerc} perc");

// Átváltás percből órára és percre
int osszPerc = 150;
int ora = osszPerc / 60;           // 2
int perc = osszPerc % 60;          // 30
Console.WriteLine($"{osszPerc} perc = {ora} óra {perc} perc");
```

Negatív számok és modulo

```
int a = -10;
int b = 3;
int maradek = a % b;           // -1 (nem 2!)
```

```
Console.WriteLine($"{a} % {b} = {maradek}");

// C#-ban a maradék előjele megegyezik az osztandó előjelével
int c = 10 % -3;           // 1
int d = -10 % -3;         // -1
```

Értékadó operátorok

Az értékadó operátorok egy változónak adnak értéket.

Alapértelmezett értékadás

```
int x = 10;                // Egyszerű értékadás
```

Összetett értékadó operátorok

Rövidebb írásmódot biztosítanak matematikai művelet + értékadás kombinációjára.

```
int szam = 10;

// Összeadás és értékadás (+=)
szam += 5;                // szam = szam + 5; → 15
Console.WriteLine(szam);

// Kivonás és értékadás (-=)
szam -= 3;                // szam = szam - 3; → 12
Console.WriteLine(szam);

// Szorzás és értékadás (*=)
szam *= 2;                // szam = szam * 2; → 24
Console.WriteLine(szam);

// Osztás és értékadás (/=)
szam /= 4;                // szam = szam / 4; → 6
Console.WriteLine(szam);

// Modulo és értékadás (%=)
szam %= 4;                // szam = szam % 4; → 2
Console.WriteLine(szam);
```

Gyakorlati példák

```
// Számláló növelése
int latogatokSzama = 0;
latogatokSzama += 1;      // Új látogató érkezett
```

```
Console.WriteLine($"Látogatók: {latogatokSzama}");

// Raktárkészlet csökkentése
int készlet = 100;
int eladottMennyiség = 15;
készlet -= eladottMennyiség; // 85
Console.WriteLine($"Raktáron maradt: {készlet} db");

// Kamat számítása
decimal egyenleg = 10000m;
decimal kamat = 0.05m; // 5%
egyenleg += egyenleg * kamat; // egyenleg = egyenleg + (egyenleg * 0.05)
Console.WriteLine($"Új egyenleg: {egyenleg} Ft"); // 10500 Ft

// String konkatenáció
string uzenet = "Hello";
uzenet += " "; // "Hello "
uzenet += "World"; // "Hello World"
Console.WriteLine(uzenet);

string nev = "János";
string koszones = "Szia, ";
koszones += nev; // "Szia, János"
koszones += "!"; // "Szia, János!"
Console.WriteLine(koszones);
```

Növelés és csökkentés operátorok

Speciális operátorok 1-gyel való növelésre/csökkentésre.

```
int x = 5;

// Post-increment (++)
int a = x++; // a = 5, x = 6 (először értéket ad, majd növel)
Console.WriteLine($"a = {a}, x = {x}");

// Pre-increment (++)
int y = 5;
int b = ++y; // b = 6, y = 6 (először növel, majd értéket ad)
Console.WriteLine($"b = {b}, y = {y}");

// Post-decrement (--)
int z = 5;
int c = z--; // c = 5, z = 4 (először értéket ad, majd csökkent)
Console.WriteLine($"c = {c}, z = {z}");

// Pre-decrement (--)
int w = 5;
int d = --w; // d = 4, w = 4 (először csökkent, majd értéket
```

```
ad)
Console.WriteLine($"d = {d}, w = {w}");
```

Fontos különbség szemléltetése:

```
// Post-increment példa
int szamlalo = 0;
Console.WriteLine(szamlalo++); // Kiírja: 0, aztán növel
Console.WriteLine(szamlalo);   // Kiírja: 1

// Pre-increment példa
int szamlalo2 = 0;
Console.WriteLine(++szamlalo2); // Növel először, kiírja: 1
Console.WriteLine(szamlalo2);   // Kiírja: 1

// Gyakorlati használat ciklusban
for (int i = 0; i < 5; i++)      // Leggyakrabban használt forma
{
    Console.WriteLine(i);       // 0, 1, 2, 3, 4
}

// Egyszerű használat
int db = 10;
db++;                          // 11 (itt mindegy pre vagy post)
Console.WriteLine(db);
```

Relációs (összehasonlító) operátorok

Két érték összehasonlítására szolgálnak. Az eredmény mindig **bool** (true vagy false).

Alapvető összehasonlítások

```
int a = 10;
int b = 20;

// Egyenlő (==)
bool egyenlo = (a == b);      // false
Console.WriteLine($"{a} == {b}: {egyenlo}");

// Nem egyenlő (!=)
bool nemEgyenlo = (a != b);   // true
Console.WriteLine($"{a} != {b}: {nemEgyenlo}");

// Kisebb (<)
bool kisebb = (a < b);        // true
Console.WriteLine($"{a} < {b}: {kisebb}");

// Nagyobb (>)
```

```
bool nagyobb = (a > b);           // false
Console.WriteLine($"{a} > {b}: {nagyobb}");

// Kisebb vagy egyenlő (<=)
bool kisebbEgyenlo = (a <= b);    // true
Console.WriteLine($"{a} <= {b}: {kisebbEgyenlo}");

// Nagyobb vagy egyenlő (>=)
bool nagyobbEgyenlo = (a >= b);   // false
Console.WriteLine($"{a} >= {b}: {nagyobbEgyenlo}");
```

FIGYELEM: == vs Equals()

```
// Értéktípusoknál (int, double, bool, stb.) a == működik
int x = 5;
int y = 5;
Console.WriteLine(x == y);        // true

// String-eknél is működik a ==
string s1 = "Hello";
string s2 = "Hello";
Console.WriteLine(s1 == s2);      // true

// De vigyázat! Objektumoknál később bonyolultabb lesz...
// (ezt majd az OOP résznél nézzük meg részletesen)
```

Gyakorlati példák

```
// Életkor ellenőrzés
int életkor = 17;
bool nagykorú = (életkor >= 18);
if (nagykorú)
{
    Console.WriteLine("Beléphet.");
}
else
{
    Console.WriteLine("Kiskorú, nem léphet be.");
}

// Jelszó hossz ellenőrzés
string jelszo = "abc123";
int minHossz = 8;
if (jelszo.Length >= minHossz)
{
    Console.WriteLine("Jelszó elfogadva.");
}
else
{
    Console.WriteLine("Jelszó nem megfelelő.");
}
```

```
        Console.WriteLine($"Jelszó túl rövid! Min. {minHossz} karakter szükséges.");
    }

    // Pontszám értékelése
    int pontszam = 85;
    if (pontszam >= 90)
    {
        Console.WriteLine("Jeles (5)");
    }
    else if (pontszam >= 80)
    {
        Console.WriteLine("Jó (4)");
    }
    else if (pontszam >= 70)
    {
        Console.WriteLine("Közepes (3)");
    }
    else if (pontszam >= 60)
    {
        Console.WriteLine("Elégséges (2)");
    }
    else
    {
        Console.WriteLine("Elégtelen (1)");
    }

    // Tartomány ellenőrzés
    int szam = 15;
    if (szam >= 10 && szam <= 20)
    {
        Console.WriteLine("A szám 10 és 20 között van.");
    }

    // Páros-páratlan ellenőrzés
    int n = 7;
    if (n % 2 == 0)
    {
        Console.WriteLine($"{n} páros");
    }
    else
    {
        Console.WriteLine($"{n} páratlan");
    }
}
```

Logikai operátorok

Logikai (bool) értékekkel végeznek műveleteket.

AND (ÉS) - && operátor

Mindkét feltételnek igaznak kell lennie.

```
bool feltetel1 = true;
bool feltetel2 = false;

bool eredmeny = feltetel1 && feltetel2; // false
Console.WriteLine(eredmeny);

// Igazságtábla
Console.WriteLine(true && true);    // true
Console.WriteLine(true && false);   // false
Console.WriteLine(false && true);   // false
Console.WriteLine(false && false);  // false
```

Gyakorlati példák:

```
// Jogosítvány + nagykorúság ellenőrzés
bool vanJogositvany = true;
bool nagykoru = true;
bool vezethetek = vanJogositvany && nagykoru;
Console.WriteLine($"Vezethetek? {vezethetek}"); // true

// Belépési feltételek
int eletkor = 25;
bool vanBelepojegy = true;
if (eletkor >= 18 && vanBelepojegy)
{
    Console.WriteLine("Bejöhhet.");
}

// Tartomány ellenőrzés
int szam = 15;
if (szam >= 10 && szam <= 20)
{
    Console.WriteLine("A szám 10 és 20 között van.");
}

// Felhasználónév és jelszó
string felhasznalonev = "admin";
string jelszo = "1234";
if (felhasznalonev == "admin" && jelszo == "1234")
{
    Console.WriteLine("Sikeres bejelentkezés!");
}
else
{
    Console.WriteLine("Hibás adatok!");
}
```

OR (VAGY) - || operátor

Elég ha legalább az egyik feltétel igaz.

```
bool feltetel1 = true;
bool feltetel2 = false;

bool eredmény = feltetel1 || feltetel2; // true
Console.WriteLine(eredmény);

// Igazságtábla
Console.WriteLine(true || true);    // true
Console.WriteLine(true || false);   // true
Console.WriteLine(false || true);   // true
Console.WriteLine(false || false);  // false
```

Gyakorlati példák:

```
// Hétvége ellenőrzés
string nap = "szombat";
if (nap == "szombat" || nap == "vasárnap")
{
    Console.WriteLine("Hétvége van!");
}

// Kedvezmény jogosultság
bool diak = false;
bool nyugdijas = true;
bool kapKedvezmenyt = diak || nyugdijas;
if (kapKedvezmenyt)
{
    Console.WriteLine("Jogosult kedvezményre.");
}

// VIP vagy admin jogosultság
bool isVIP = false;
bool isAdmin = true;
if (isVIP || isAdmin)
{
    Console.WriteLine("Hozzáférés engedélyezve.");
}

// Fizetési módok
string fizetesMod = "kártya";
if (fizetesMod == "készpénz" || fizetesMod == "kártya" || fizetesMod == "átutalás")
{
    Console.WriteLine("Elfogadott fizetési mód.");
}
```

NOT (NEM) - ! operátor

Megfordítja a logikai értéket.

```
bool igaz = true;
bool hamis = !igaz;           // false
Console.WriteLine(hamis);

// Példák
Console.WriteLine(!true);     // false
Console.WriteLine(!false);    // true

bool esikAz = false;
bool nemEsikAz = !esikAz;     // true
Console.WriteLine($"Nem esik az eső? {nemEsikAz}");
```

Gyakorlati példák:

```
// Bejelentkezési állapot
bool bejelentkezve = false;
if (!bejelentkezve)
{
    Console.WriteLine("Kérem jelentkezzen be!");
}

// Hiányzó adat ellenőrzés
string email = "";
if (!(email.Length > 0)) // vagy: if (email.Length == 0)
{
    Console.WriteLine("Email megadása kötelező!");
}

// Kapcsoló állapot
bool lampaBekapcsolva = true;
lampaBekapcsolva = !lampaBekapcsolva; // false (kikapcsolás)
Console.WriteLine($"Lámpa állapota: {lampaBekapcsolva}");

// Dupla negáció
bool aktiv = true;
bool eredmény = !aktiv;           // true (nem változik)
Console.WriteLine(eredmény);
```

Összetett logikai kifejezések

```
// Zárójelek használata
bool a = true;
bool b = false;
bool c = true;

bool eredmény1 = a && b || c;           // (true && false) || true = true
```

```
bool eredmeny2 = a && (b || c);           // true && (false || true) = true
bool eredmeny3 = (a || b) && c;           // (true || false) && true = true

Console.WriteLine($"eredmeny1: {eredmeny1}");
Console.WriteLine($"eredmeny2: {eredmeny2}");
Console.WriteLine($"eredmeny3: {eredmeny3}");

// Komplex feltétel példa
int életkor = 25;
bool vanJogsi = true;
bool vanAuto = false;
bool elerhető = true;

// Vezethet, ha nagykorú ÉS van jogosítványa ÉS (van autója VAGY elérhető egy)
bool vezethet = (életkor >= 18) && vanJogsi && (vanAuto || elerhető);
Console.WriteLine($"Vezethet? {vezethet}");

// Belépési feltételek
string jegyTipus = "VIP";
int jegyAr = 5000;
bool vanKedvezmeny = true;

// Beléphet, ha (VIP jegye van VAGY fizette a teljes árat) ÉS megvan a jegye
bool belephet = (jegyTipus == "VIP" || (jegyAr >= 5000 && !vanKedvezmeny)) &&
jegyTipus != "";
Console.WriteLine($"Beléphet? {belephet}");
```

Rövid-kiértékelés (Short-circuit evaluation)

A C# hatékonysági okokból **nem értékeli ki** a teljes kifejezést, ha már az első rész alapján eldönthető az eredmény.

```
// && operátornál
bool a = false;
bool b = true;

// Ha az első false, a másodikat már nem nézi meg
if (a && b) // a false, ezért b-t már nem értékeli ki
{
    Console.WriteLine("Nem fut le");
}

// Gyakorlati példa - biztonságos null ellenőrzés
string nev = null;
if (nev != null && nev.Length > 0) // Ha nev null, a nev.Length-et már nem vizsgálja
{
    Console.WriteLine("Van név");
}
// Ha nem lenne rövid-kiértékelés, NullReferenceException lenne!
```

```
// || operátornál
bool x = true;
bool y = false;

// Ha az első true, a másodikat már nem nézi meg
if (x || y) // x true, ezért y-t már nem értékeli ki
{
    Console.WriteLine("Lefut"); // Lefut
}

// Gyakorlati példa
string input = "";
if (input == null || input.Length == 0) // Ha input null, a Length-et nem vizsgálja
{
    Console.WriteLine("Üres input");
}
```

Operátorok precedenciája (műveleti sorrend)

Az operátorok végrehajtási sorrendje fix, de zárójelekkel felülírható.

Prioritási sorrend (magasabbtól alacsonyabb felé)

1. **Zárójelek:** `()`
2. **Prefix növelés/csökkentés:** `++, --, !`
3. **Szorzás, osztás, modulo:** `*, /, %`
4. **Összeadás, kivonás:** `+, -`
5. **Relációs operátorok:** `<, >, <=, >=`
6. **Egyenlőség:** `==, !=`
7. **Logikai ÉS:** `&&`
8. **Logikai VAGY:** `||`
9. **Értékdadás:** `=, +=, -=, stb.`

Példák

```
// Matematikai műveletek
int eredmeny1 = 2 + 3 * 4;           // 14 (nem 20!) - előbb szorzás
int eredmeny2 = (2 + 3) * 4;         // 20 - zárójelek először

int eredmeny3 = 10 + 20 / 2;         // 20 (nem 15!)
int eredmeny4 = (10 + 20) / 2;       // 15

// Relációs és logikai
bool eredmeny5 = 5 > 3 && 10 < 20;   // true && true = true
bool eredmeny6 = 5 > 3 || 10 > 20;   // true || false = true
bool eredmeny7 = !(5 > 3);           // !true = false

// Komplex kifejezés
```

```
int x = 5;
bool eredmeny8 = x > 3 && x < 10 || x == 0;
// (5 > 3 && 5 < 10) || 5 == 0
// (true && true) || false
// true || false
// true

// Ugyanaz zárójelekkel egyértelműbben
bool eredmeny9 = (x > 3 && x < 10) || (x == 0);

// Értékadás precedenciája
int a = 5;
int b = 10;
int c = a + b * 2;           // 25 (nem 30!)
int d = (a + b) * 2;         // 30

// Gyakorlati példa
int ar = 1000;
int mennyiseg = 5;
double afa = 1.27;
double vegosszeg = ar * mennyiseg * afa; // ((ar * mennyiseg) * afa)
Console.WriteLine(vegosszeg);           // 6350
```

Tipp: Használj zárójeleket az egyértelműség kedvéért!

```
// Nehezen olvasható
bool eredmeny = életkor >= 18 && jogositvany == true || vanKiserő == true;

// Egyértelmű zárójelekkel
bool eredmeny2 = (életkor >= 18 && jogositvany) || vanKiserő;
```

Egyéb hasznos operátorok

Ternáris (feltételes) operátor - ? :

Egysoros if-else alternatíva.

Szintaxis: `feltétel ? igaz_esetén : hamis_esetén`

```
int életkor = 20;

// Hagyományos if-else
string eredmeny1;
if (életkor >= 18)
{
    eredmeny1 = "nagykorú";
}
else
```

```

{
    eredmény1 = "kiskorú";
}

// Ternáris operátorral (rövidebb)
string eredmény2 = (eletkor >= 18) ? "nagykorú" : "kiskorú";
Console.WriteLine(eredmény2); // nagykorú

// További példák
int szam = 15;
string paritate = (szam % 2 == 0) ? "páros" : "páratlan";
Console.WriteLine($"{szam} {paritate}"); // 15 páratlan

// Maximum kiválasztás
int a = 10;
int b = 20;
int max = (a > b) ? a : b;
Console.WriteLine($"Maximum: {max}"); // 20

// Kedvezmény számítása
bool vipTag = true;
decimal ar = 10000m;
decimal fizetendo = vipTag ? ar * 0.9m : ar; // 10% kedvezmény VIP-nek
Console.WriteLine($"Fizetendő: {fizetendo} Ft");

// Üdvözlő üzenet
string nev = "Anna";
string udvozles = (nev != "") ? $"Szia, {nev}!" : "Szia!";
Console.WriteLine(udvozles);

// Beágyazott ternáris (óvatosan használd!)
int pontszam = 85;
string osztalyzat = (pontszam >= 90) ? "jeles" :
                    (pontszam >= 80) ? "jó" :
                    (pontszam >= 70) ? "közepes" :
                    (pontszam >= 60) ? "elégséges" : "elégtelen";
Console.WriteLine(osztalyzat); // jó

```

Null-coalescing operátor - ??

Ha az első érték null, akkor a másodikat adja vissza.

```

string nev = null;
string megjelenítendőNev = nev ?? "Névtelen";
Console.WriteLine(megjelenítendőNev); // Névtelen

// Láncolható
string becenev = null;
string keresztnév = null;
string vezeteknev = "Nagy";
string használandóNev = becenev ?? keresztnév ?? vezeteknev ?? "Ismeretlen";

```

```
Console.WriteLine(használandóNev); // Nagy

// Alapértelmezett érték beállítása
string beallitas = null;
string ertekek = beallitas ?? "alapértelmezett";
```

Null-conditional operátor - ?.

Biztonságos hozzáférés tulajdonságokhoz és metódusokhoz.

```
string nev = null;

// Hagyományos null ellenőrzés
int? hossz1 = null;
if (nev != null)
{
    hossz1 = nev.Length;
}

// Null-conditional operátorral (rövidebb)
int? hossz2 = nev?.Length; // null, mert nev null
Console.WriteLine(hossz2);

// Kombináció null-coalescing-gel
string szoveg = null;
int hossz3 = szoveg?.Length ?? 0; // Ha null, akkor 0
Console.WriteLine(hossz3); // 0

// Metódus hívása
string input = null;
string nagybetus = input?.ToUpper(); // null, nem dob kivételt
Console.WriteLine(nagybetus ?? "nincs adat");
```

Összefoglaló gyakorlat

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("=== Számológép ===\n");

        // Bemenet
        Console.Write("Add meg az első számot: ");
        double szam1 = Convert.ToDouble(Console.ReadLine());

        Console.Write("Add meg a műveletet (+, -, *, /, %): ");
        string muvelet = Console.ReadLine();
```

```
Console.Write("Add meg a második számot: ");
double szam2 = Convert.ToDouble(Console.ReadLine());

// Számítás
double eredmeny = 0;
bool ervenyesMuvelet = true;

if (muvelet == "+")
{
    eredmeny = szam1 + szam2;
}
else if (muvelet == "-")
{
    eredmeny = szam1 - szam2;
}
else if (muvelet == "*")
{
    eredmeny = szam1 * szam2;
}
else if (muvelet == "/")
{
    if (szam2 != 0)
    {
        eredmeny = szam1 / szam2;
    }
    else
    {
        Console.WriteLine("HIBA: Nullával nem lehet osztani!");
        ervenyesMuvelet = false;
    }
}
else if (muvelet == "%")
{
    if (szam2 != 0)
    {
        eredmeny = szam1 % szam2;
    }
    else
    {
        Console.WriteLine("HIBA: Nullával nem lehet osztani!");
        ervenyesMuvelet = false;
    }
}
else
{
    Console.WriteLine("HIBA: Érvénytelen művelet!");
    ervenyesMuvelet = false;
}

// Kimenet
if (ervenyesMuvelet)
{
    Console.WriteLine($"Eredmény: {szam1} {muvelet} {szam2} = {eredmeny}");
}
```



```
// Ellenőrzések
bool pozitiv = eredmeny > 0;
bool negativ = eredmeny < 0;
bool nulla = eredmeny == 0;

Console.WriteLine($"Pozitív? {pozitiv}");
Console.WriteLine($"Negatív? {negativ}");
Console.WriteLine($"Nulla? {nulla}");

// Ternáris operátorral
string tipus = (eredmeny > 0) ? "pozitív" :
               (eredmeny < 0) ? "negatív" : "nulla";
Console.WriteLine($"Az eredmény {tipus}.");
    }
}
```

Összefoglalás

Aritmetikai operátorok

- `+` összeadás
- `-` kivonás
- `*` szorzás
- `/` osztás
- `%` maradékos osztás (modulo)

Értékadó operátorok

- `=` egyszerű értékadás
- `+=`, `-=`, `*=`, `/=`, `%=` összetett értékadás
- `++`, `--` növelés/csökkentés

Relációs operátorok

- `==` egyenlő
- `!=` nem egyenlő
- `<` kisebb
- `>` nagyobb
- `<=` kisebb vagy egyenlő
- `>=` nagyobb vagy egyenlő

Logikai operátorok

- `&&` ÉS (AND)
- `||` VAGY (OR)
- `!` NEM (NOT)

Speciális operátorok

- `?:` ternáris (feltételes)
- `??` null-coalescing
- `?.` null-conditional

Fontos:

- Operátorok precedenciája meghatározza a műveleti sorrendet
- Zárójelek használata növeli az olvashatóságot
- Egész osztás csonkít, nem kerekít
- Rövid-kiértékelés a logikai műveleteknél

C# Alapok Tananyag - 3. Fejezet